

JAVA PROJECT

COMPUTER SCIENCE & ENGINEERING -
(Artificial Intelligence & Machine Learning)

Submitted by -

24WH1A6604-Ms. B. RASHMITHA

24WH1A6616-Ms. S. NIKITHA

24WH1A639- Ms. J. DHATHRI

24WH1A6641-Ms. J. GRACY PRASANNA

24WH1A6642-Ms. K. NITHYA

Under the esteemed guidance of

Dr. B Lakshmi Praveena

Professor

CSE (AI & ML)



BVRIT HYDERABAD COLLEGE OF ENGINEERING FOR WOMEN

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Accredited by NAAC with A Grade

Bachupally, Hyderabad – 500090

BVRIT HYDERABAD
COLLEGE OF ENGINEERING FOR WOMEN

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)
Accredited by NAAC with A Grade
Bachupally, Hyderabad – 500090

Department of
Computer Science & Engineering -
(Artificial Intelligence & Machine Learning)



CERTIFICATE

This is to certify that the **Java Project** is a Bonafide work carried out by 24WH1A6604 Ms. B. Rashmitha, 24WH1A6616 Ms. S. Nikitha, 24WH1A6639 Ms. J. Dhathri, 24WH1A6641 Ms. J. Gracy Prasanna, 24WH1A6642 Ms. K. Nithya in partial fulfilment for the award of B.Tech degree **in Computer Science & Engineering (AI & ML), BVRIT HYDERABAD College of Engineering for Women, Bachupally, Hyderabad**, affiliated to Jawaharlal Nehru Technological University Hyderabad, under my guidance and supervision. The results embodied in the project work have not been submitted to any other.

Faculty of the Department
Dr. B Lakshmi Praveena
Professor
Dept of CSE(AI&ML)

Head of the Department
Dr. B Lakshmi Praveena
HOD & Professor
Dept of CSE(AI&ML)

INDEX

S.no	Contents	Page no	Remarks
1.	Abstract	4	
2.	Introduction	5	
3.	Literature Review	6	
4.	Technology Stack	7	
5.	Implementation(code)	8-9	
6.	Output of the Code	10	
7.	Conclusion	11	
8.	Reference	12	

ABSTRACT:

The Simple Chat Application is a Java-based desktop application developed using the Swing framework.

The main objective of this project is to simulate a basic chat environment where messages can be displayed along with the sender's name.

A drop-down menu allows the user to select different names, giving the appearance of different people sending messages.

The application demonstrates core concepts of Java GUI development, event handling, and component interaction.

This project is simple, lightweight, and ideal for understanding how chat interfaces work at a basic level.

INTRODUCTION:

Communication through chat applications has become an integral part of modern digital interaction.

Although real chat systems use networking and server-client communication, this project focuses on the graphical interface aspect of a chat system.

The goal of the Simple Chat Application is to create a chat-like window where the user can type a message, select a sender name from a dropdown list, and display the message in the chat area.

The application uses **Java Swing**, which provides robust components for building graphical user interfaces.

This project helps beginners understand:

- GUI development
- Layout management in Java
- Event-driven programming using ActionListeners
- Working with Swing components like JComboBox, JTextArea, JTextField, and JButton

The simplicity of this project makes it ideal for learning the foundations of Java UI programming.

LITERATURE REVIEW:

Java Swing has long been used as a standard toolkit for designing graphical user interfaces. Literature on Java GUI development highlights the following:

Swing Framework

Swing is a part of Java Foundation Classes (JFC) and offers:

- Platform-independent GUI components
- A lightweight architecture
- MVC-based component design

Swing components like JTextArea, JButton, and JComboBox provide flexibility to design custom user interfaces.

Event Handling

Event-driven programming plays a crucial role in GUIs. Java's event-handling model allows components to react to events such as button clicks or typing. This project uses the **ActionListener interface** to detect the "Send" button action.

Use of Layout Managers

Literature emphasizes the importance of layout managers for well-structured UI. This project uses **BorderLayout**, which organizes components efficiently across the frame.

Chat Simulation

While real chat applications require networking and threads, simulated chat interfaces help learners understand:

- Message display logic
- Component interaction
- UI flow of chat-based systems

TECHNOLOGY STACK:

Programming Language

- **Java** (JDK 8+) Used to develop and run the application.

Libraries and Frameworks

- **Java** **Swing** Used to build the GUI components like buttons, text areas, and combo boxes.

Tools Used

- **IDE:** Eclipse /Terminal (Ubuntu)/ VS Code (Windows & Ubuntu)
- **JVM:** To execute the compiled Java program

Concepts Used

- Object-Oriented Programming
- Event Handling
- GUI Development
- Layout Management

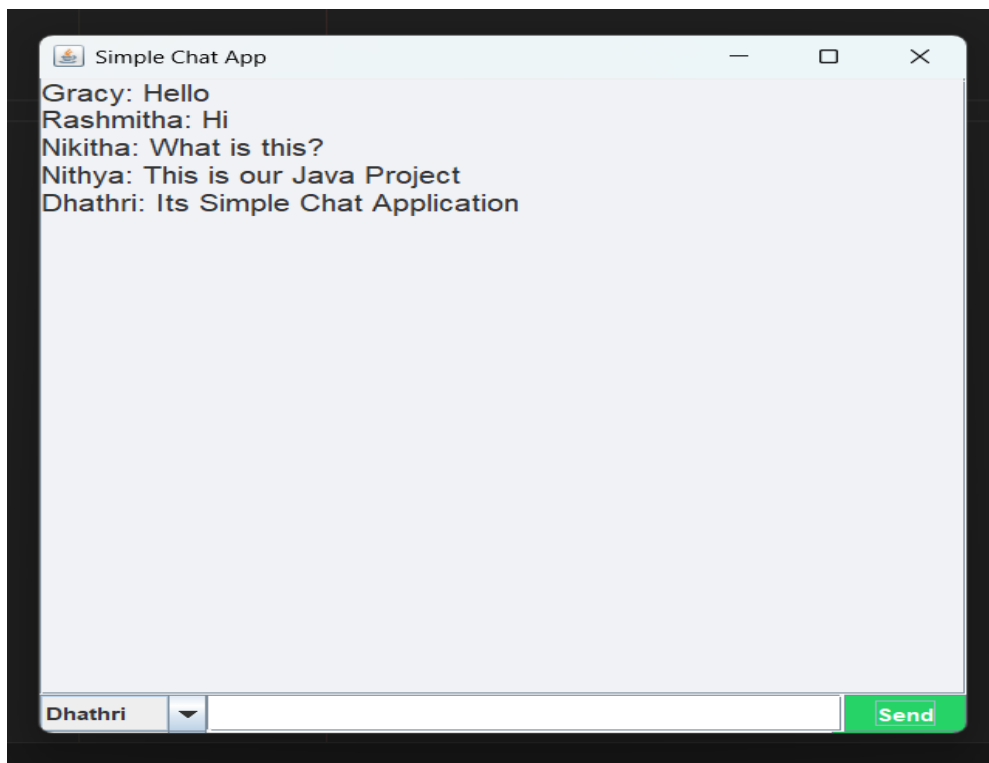
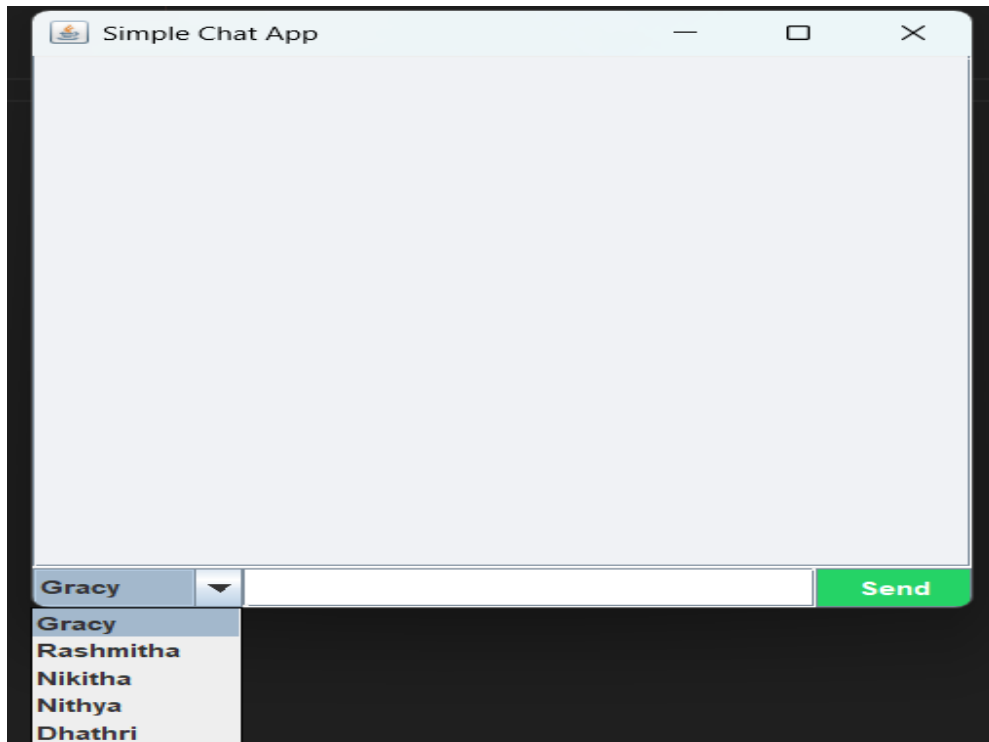
IMPLEMENTATION(CODE):

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class SimpleChatApplication extends JFrame implements ActionListener
{
    JComboBox users;
    JTextArea chatArea;
    JTextField textField;
    JButton sendButton;
    public SimpleChatApplication() {
        setTitle("Simple Chat App");
        setSize(400, 400);
        setLayout(new BorderLayout());
        chatArea = new JTextArea();
        chatArea.setEditable(false);
        chatArea.setBackground(new Color(240, 242, 245));
        chatArea.setFont(new Font("Arial", Font.PLAIN, 16));
        add(new JScrollPane(chatArea), BorderLayout.CENTER);
        JPanel panel = new JPanel();
        panel.setLayout(new BorderLayout());
        panel.setBackground(Color.WHITE);
        String names[] = {"Gracy", "Rashmitha", "Nikitha", "Nithya", "Dhathri"};
        users = new JComboBox(names);
        panel.add(users, BorderLayout.WEST);
        textField = new JTextField();
        panel.add(textField, BorderLayout.CENTER);
        sendButton = new JButton("Send");
        sendButton.setBackground(new Color(37, 211, 102));
```



```
        sendButton.setForeground(Color.WHITE);
        sendButton.addActionListener(this);
        panel.add(sendButton, BorderLayout.EAST);
        add(panel, BorderLayout.SOUTH);
        getContentPane().setBackground(new Color(180, 200, 255));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        String user = (String) users.getSelectedItem();
        String msg = textField.getText();
        if (!msg.equals("")) {
            chatArea.append(user + ": " + msg + "\n");
            textField.setText("");
        }
    }
    public static void main(String[] args) {
        new SimpleChatApplication();
    }
}
```

OUTPUT:



CONCLUSION:

The Simple Chat Application successfully demonstrates the fundamental concepts of Java GUI development using Swing. Users can simulate chat messages by selecting different names from a dropdown list and sending text to the chat window. Although it does not use networking, the project serves as a strong foundation for building real-world chat applications in the future. This project enhances understanding of event handling, interface design, and user interaction models in Java.

REFERENCES:

1. Herbert Schildt, **Java: The Complete Reference**, McGraw-Hill Education
2. Java Documentation –
Swing:
<https://docs.oracle.com/javase/8/docs/technotes/guides/swing/>
3. Deitel & Deitel, **Java: How to Program**
4. Tutorials Point – Java Swing Basics
5. GeeksforGeeks – Java GUI Programming